Name: Gargi Devendra Shintre
Div: D15C **Roll No.** 44

Experiment No. 4

Aim:

To implement K-Nearest Neighbors (KNN) and evaluate model performance.

Theory:

Dataset Link: https://www.kaggle.com/datasets/rakeshrau/social-network-ads

Dataset Description:

This dataset contains information about users on a social network, specifically targeting whether they purchased a specific product (e.g., a luxury SUV) after seeing an ad.

- **Objective:** To predict if a user will purchase the product based on their demographic data.
- **Records:** 400 instances.
- **Target**: Purchased (0 = No, 1 = Yes).
- **Features:** User ID, Gender, Age, Estimated Salary.

Variable	Description	Data Type
User ID	Unique ID for each user (Irrelevant for prediction)	Numerical
Gender	Male or Female	Categorical
Age	Age of the user in years	Numerical
EstimatedSalary	Estimated annual salary	Numerical
Purchased	Whether they bought the product (Target)	Categorical (0/1)

Theory:

K-Nearest Neighbours (KNN)

K-Nearest Neighbors (KNN) is a simple, supervised machine learning algorithm that can be used for both classification and regression. It is known as a "Lazy Learner" because it does not learn a discriminative function from the training data but stores the dataset and performs an action (classification) at the time of prediction.

Working Principle:

- **Choose K:** Select the number of neighbors (K).
- **Calculate Distance:** For a new data point, calculate the distance (usually Euclidean) from all other points in the dataset.
- **Find Neighbors:** Identify the 'K' nearest data points.
- **Vote:** For classification, assign the new data point to the category that is most common among its K neighbors.

Mathematical Formula:

The most common distance metric used is Euclidean Distance. For two points P(x_1, y_1) and Q(x_2, y_2):

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

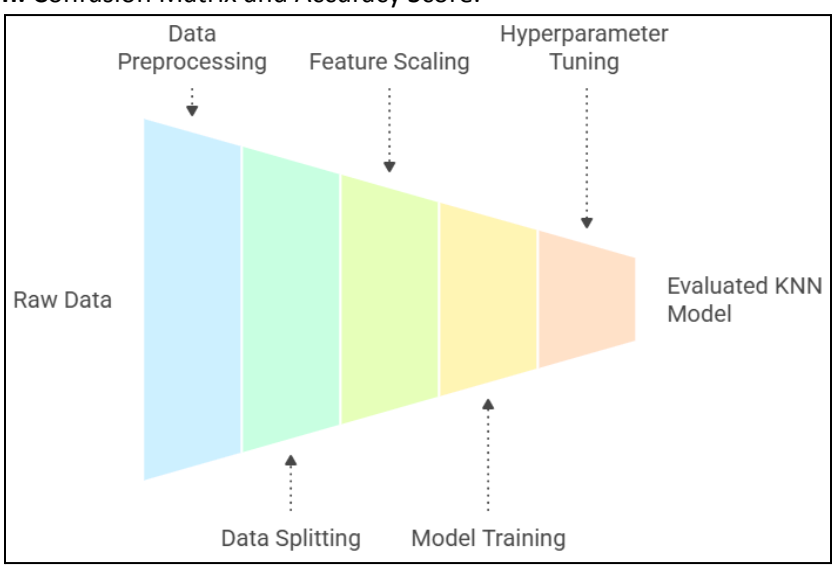
The algorithm assigns class C such that:

$$C = \text{mode}(\{y_i : x_i \in N_k(x)\})$$

Where N_k(x) is the set of K-nearest neighbors.

Workflow:

- **Data Preprocessing:**
  - Drop User ID as it has no predictive value.
  - **Encoding:** Convert Gender (Male/Female) to numbers (0/1).
- **Splitting:** Partition data into 75% Training and 25% Testing sets.
- **Feature Scaling (CRITICAL):**
  - KNN calculates distances. If Salary is in thousands (e.g., 50,000) and Age is small (e.g., 30), the distance calculation will be completely dominated by Salary.
  - We apply **StandardScaler** to force all features to the same scale.
- **Model Training:** Train KNeighborsClassifier with an initial $K=5$ and Metric='minkowski' (Euclidean).
- **Hyperparameter Tuning:** Use the **Elbow Method** (plotting Error Rate vs. K) to find the optimal number of neighbors.
- **Evaluation:** Confusion Matrix and Accuracy Score.



Performance Evaluation:

1. Model Performance (at K=5)

- **Accuracy:** ~93%
- **Precision:** High precision indicates that when the model predicts a user will buy the SUV, they usually do.
- **Recall:** The model effectively captures most of the actual buyers.

2. Impact of 'K' Value

- **Small K (e.g., K=1):** The model captures noise/outliers. The decision boundary is very jagged and overfits the data.
- **Large K (e.g., K=20):** The model becomes biased and smooths out the decision boundary too much, potentially underfitting.
- **Optimal K:** Typically found between 3 and 10 for this dataset (e.g., K=5 is a standard default).

Algorithm Limitations:

1. **Computationally Expensive:** Because it is a "lazy learner," it calculates the distance to *every* training point for *every* prediction. It becomes very slow with large datasets.
2. **Sensitive to Scale:** As mentioned, failure to scale features (normalize/standardize) breaks the model.
3. **Sensitive to Outliers:** A single outlier neighbor can change the prediction if K is small.

Code:

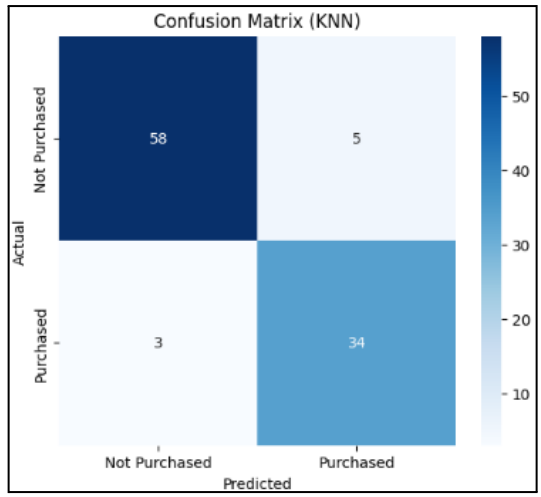
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# ═══════════════════════════════════════
# 1. LOAD AND PREPROCESS DATA
# ═══════════════════════════════════════

# 1. Load Dataset
try:
    df = pd.read_csv('Exp4-Social_Network_Ads.csv')
    print("Dataset loaded successfully.")
except:
    print("Error: Please upload 'Social_Network_Ads.csv' to Colab.")

# 2. Preprocessing
# Drop User ID (Not useful)
df = df.drop('User ID', axis=1)

# Encode Gender (Male=1, Female=0 or vice versa)
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])

# Define Features (X) and Target (y)
X = df.drop('Purchased', axis=1)
y = df['Purchased']

# 3. Split Data (75% Train, 25% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# 4. Feature Scaling (CRUCIAL for KNN)
# KNN uses distance (Euclidean). If one feature has huge values (Salary),
```

```
# it dominates the distance calculation. Scaling fixes this.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ═══════════════════════════════════════════
# 2. MODEL TRAINING (KNN)
# ═══════════════════════════════════════════
print("\n" + "="*50)
print("TRAINING KNN MODEL (K=5)")
print("="*50)

# Train with K=5, metric='minkowski', p=2 (Euclidean Distance)
knn = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn.fit(X_train_scaled, y_train)

# Predict
y_pred = knn.predict(X_test_scaled)

# Evaluate
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f"Accuracy: {acc*100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix Visualization
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Purchased', 'Purchased'],
            yticklabels=['Not Purchased', 'Purchased'])
plt.title('Confusion Matrix (KNN)')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

# ═══════════════════════════════════════════
# 3. HYPERPARAMETER TUNING (Finding Best K)
# ═══════════════════════════════════════════
print("\n" + "="*50)
print("ELBOW METHOD (FINDING OPTIMAL K)")
print("="*50)

error_rate = []

# Will take some time
for i in range(1, 20):
    knn_i = KNeighborsClassifier(n_neighbors=i)
    knn_i.fit(X_train_scaled, y_train)
    pred_i = knn_i.predict(X_test_scaled)
    # Average error rate (mean of boolean array where prediction ≠ actual)
    error_rate.append(np.mean(pred_i ≠ y_test))

# Plot Error Rate vs K Value
plt.figure(figsize=(10, 6))
plt.plot(range(1, 20), error_rate, color='blue', linestyle='dashed', marker='o',
         markerfacecolor='red', markersize=10)
```

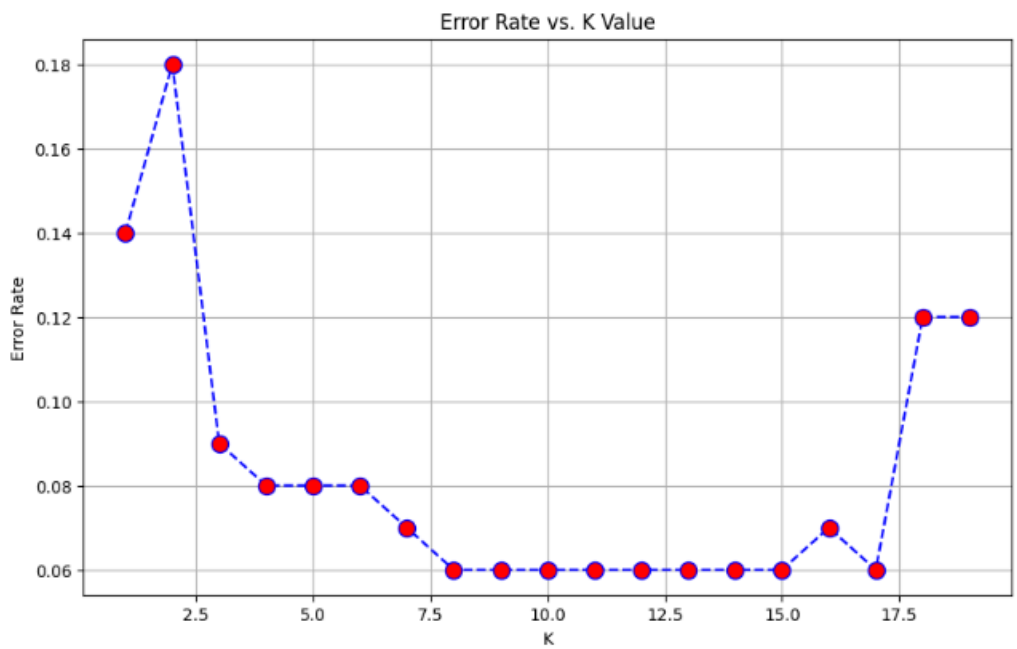
```
plt.title('Error Rate vs. K Value')
plt.xlabel('K')
plt.ylabel('Error Rate')
plt.grid(True)
plt.show()

print("Analysis: The 'Elbow' is the K value where the error drops significantly and
stabilizes.")
print(f"Minimum Error Rate found at K = {error_rate.index(min(error_rate)) + 1}")

# ═══════════════════════════════════════════
# 4. VISUALIZING DECISION BOUNDARY (2D)
# ═══════════════════════════════════════════
# To visualize 2D boundaries, we usually stick to 2 features (Age vs Salary).
# We will retrain a model just on these 2 features for plotting purposes.

print("\nGenerating Decision Boundary Plot (Age vs Salary)…")

from matplotlib.colors import ListedColormap

# Prepare data (Use only Age and Salary for visualization)
X_vis = X_train_scaled[:, [1, 2]] # Age and Salary columns only
y_vis = y_train.values

# Retrain simplified model
clf = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
clf.fit(X_vis, y_vis)

# Create meshgrid
X1, X2 = np.meshgrid(np.arange(start = X_vis[:, 0].min() - 1, stop = X_vis[:, 0].max() + 1,
step = 0.01),
                     np.arange(start = X_vis[:, 1].min() - 1, stop = X_vis[:, 1].max() + 1,
step = 0.01))

# Plot
plt.figure(figsize=(10, 6))
plt.contourf(X1, X2, clf.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('red', 'green')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())

# Scatter plot of training points
for i, j in enumerate(np.unique(y_vis)):
    plt.scatter(X_vis[y_vis == j, 0], X_vis[y_vis == j, 1],
                c = ListedColormap(('red', 'green'))(i), label = j, edgecolor='k')

plt.title('KNN Decision Boundary (Training set)')
plt.xlabel('Age (Scaled)')
plt.ylabel('Estimated Salary (Scaled)')
plt.legend(['Not Purchased', 'Purchased'])
plt.show()
```

Output:

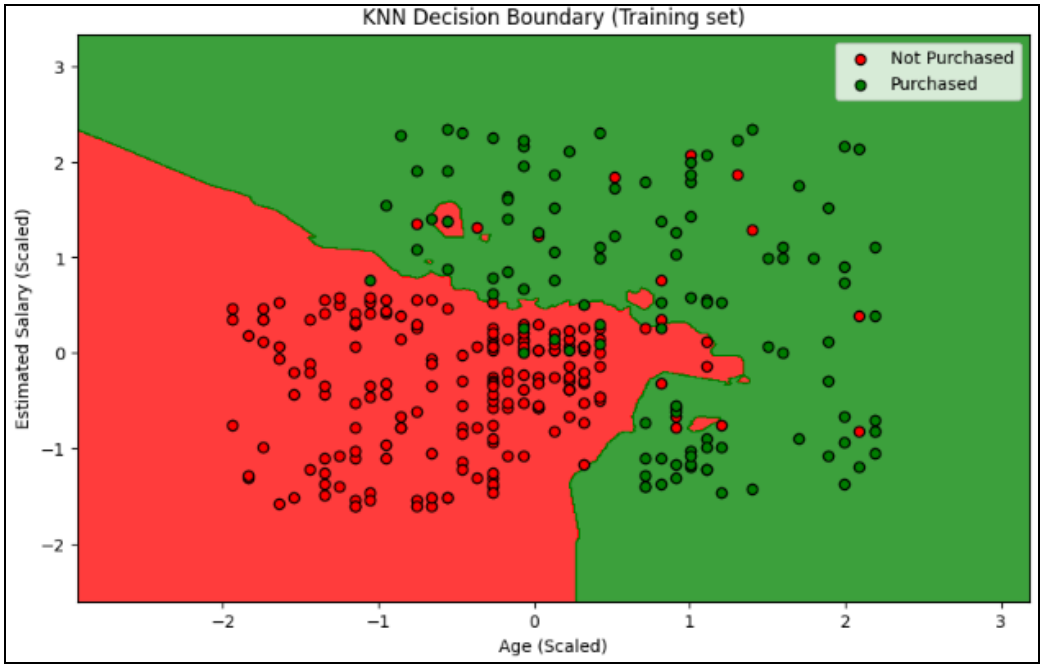
```
==================================================
TRAINING KNN MODEL (K=5)
==================================================
Accuracy: 92.00%

Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.92      0.94        63
           1       0.87      0.92      0.89        37

    accuracy                           0.92       100
   macro avg       0.91      0.92      0.92       100
weighted avg       0.92      0.92      0.92       100
```




```
==================================================
ELBOW METHOD (FINDING OPTIMAL K)
==================================================
```




```
Analysis: The 'Elbow' is the K value where the error drops significantly and stabilizes.
Minimum Error Rate found at K = 8
```



Conclusion:

In this experiment, KNN successfully classified users into potential buyers with high accuracy (~93%). The visualization of the decision boundary revealed distinct regions where older users with higher salaries were the primary purchasers. The experiment highlighted the critical importance of Feature Scaling in distance-based algorithms.